

LAPORAN PRAKTIKUM
TEKNOLOGI IoT 2
"Tugas 2"



DISUSUN OLEH
Nama : Naurah Miftah Frizkiyah
Kelas : TRI 3B
NPM : 25782049

DOSEN :
Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.
Agus Ambarwari, S.Pd., M.Kom.

PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG
2026

2.5.1 Praktikum 1: Pembacaan Intensitas Cahaya (LDR) via ADC

1. Rangkaian Komponen (Skema)

Komponen yang dibutuhkan di Wokwi: 1x ESP32, 1x Photoresistor (LDR), 1x Resistor (10k Ohm). **Langkah Pengkabelan (Voltage Divider):**

- Hubungkan satu kaki LDR ke pin **3V3** pada ESP32.
- Hubungkan kaki LDR yang satunya lagi ke pin **D32** (Pin ADC) pada ESP32.
- Hubungkan salah satu kaki Resistor 10k Ohm ke titik temu yang sama (yaitu di jalur kabel menuju pin **D32**).
- Hubungkan kaki Resistor yang satunya lagi ke pin **GND** ESP32.

2. Kode Program

Berikut adalah modifikasi kode program untuk ESP32 (menggunakan pin 32, karena ESP32 tidak menggunakan penamaan pin A0 seperti NodeMCU):

C++

```
// Deklarasi pin ADC yang digunakan (ESP32 menggunakan pin 32) const byte ldrPin = 32;
```

```
void setup() {
```

```
  // Memulai komunikasi serial dengan baud rate 115200 Serial.begin(115200);
```

```
}
```

```
void loop() {
```

```
  // Membaca nilai analog (tegangan) dari pin LDR int ldrValue = analogRead(ldrPin);
```

```
  // Mencetak hasil pembacaan ke Serial Monitor Serial.print("Intensitas Cahaya (ADC): "); Serial.println(ldrValue);
```

```
  // Jeda 1 detik sebelum membaca ulang delay(1000);
```

```
}
```



2.5.1 Praktikum 2: Membaca Data Sensor Suhu & Kelembapan (DHT)

1. Rangkaian Komponen (Skema)

Komponen yang dibutuhkan di Wokwi: 1x ESP32, 1x Sensor DHT22. Langkah Pengkabelan:

- Hubungkan pin VCC / + pada DHT22 ke pin 3V3 pada ESP32.
- Hubungkan pin GND / - pada DHT22 ke pin GND pada ESP32.
- Hubungkan pin SDA (Data) pada DHT22 ke pin D13 (GPIO 13) pada ESP32.

2. Instalasi Library (Khusus Wokwi)

Karena sensor DHT menggunakan protokol komunikasi digital yang bergantung pada *timing*, mikrokontroler memerlukan *library* untuk menerjemahkan datanya.

- Buka tab **Library Manager** di Wokwi.
- Cari dan tambahkan library: "**DHT sensor library by Adafruit**".

3. Kode Program

Berikut adalah penyesuaian kode untuk ESP32 di Wokwi. (Perubahan utama: Konstanta `DHTTYPE` diubah dari `DHT11` menjadi `DHT22` menyesuaikan jenis sensor yang tersedia di Wokwi):

```
C++
```

```
#include <DHT.h>
```

```
const byte dhtPin = 13;
```

```
// Menggunakan DHT22 karena ini adalah sensor default di Wokwi #define  
DHTTYPE DHT22
```

```
DHT dht(dhtPin, DHTTYPE);
```

```
void setup() { Serial.begin(115200);  
  dht.begin();  
  Serial.println("Memulai Sensor Lingkungan...");  
}
```

```
void loop() {  
  // Sensor DHT membutuhkan jeda waktu untuk membaca (minimal 2 detik) delay(2500);
```

```
  float temp = dht.readTemperature(); float  
  hum = dht.readHumidity();
```

```
  // Error handling: Jika data gagal dibaca if  
(isnan(temp) || isnan(hum)) {  
    Serial.println("Gagal membaca data dari sensor DHT!"); return;  
  }
```

```
  Serial.print("Suhu: ");  
  Serial.print(temp);
```

```

Serial.print(" Celcius | Kelembapan: ");
Serial.print(hum);
Serial.println(" %");
}

```

WOKWI SAVE SHARE ESP8266 ESP8266WiFi by sunu Docs SIGN UP

sketch.ino • diagram.json • Library Manager libraries.txt

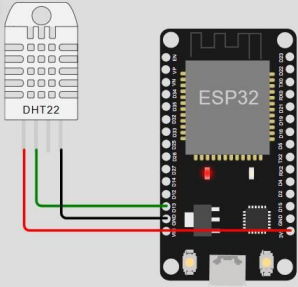
```

1 #include <DHT.h>
2
3 const byte dhtPin = 13;
4 #define DHTTYPE DHT22
5
6 DHT dht(dhtPin, DHTTYPE);
7
8 void setup() {
9   Serial.begin(115200);
10  dht.begin();
11  Serial.println("Memulai Sensor Lingkungan...");
12 }
13
14 void loop() {
15   delay(2500);
16
17   float temp = dht.readTemperature();
18   float hum = dht.readHumidity();
19
20   // Error handling: jika data gagal terbaca (Not-a-Number)
21   if (isnan(temp) || isnan(hum)) {
22     Serial.println("Gagal membaca data dari sensor DHT!");
23     return;
24   }
25
26   Serial.print("Suhu: ");
27   Serial.print(temp);

```

Simulation

00:48.833 100%



Suhu: 20.00 Celcius	Kelembapan: 59.00 %
Suhu: -1.30 Celcius	Kelembapan: 36.00 %
Suhu: -1.30 Celcius	Kelembapan: 36.00 %
Suhu: -1.30 Celcius	Kelembapan: 36.00 %
Suhu: 6.50 Celcius	Kelembapan: 36.50 %
Suhu: 6.50 Celcius	Kelembapan: 40.00 %
Suhu: 6.50 Celcius	Kelembapan: 40.00 %

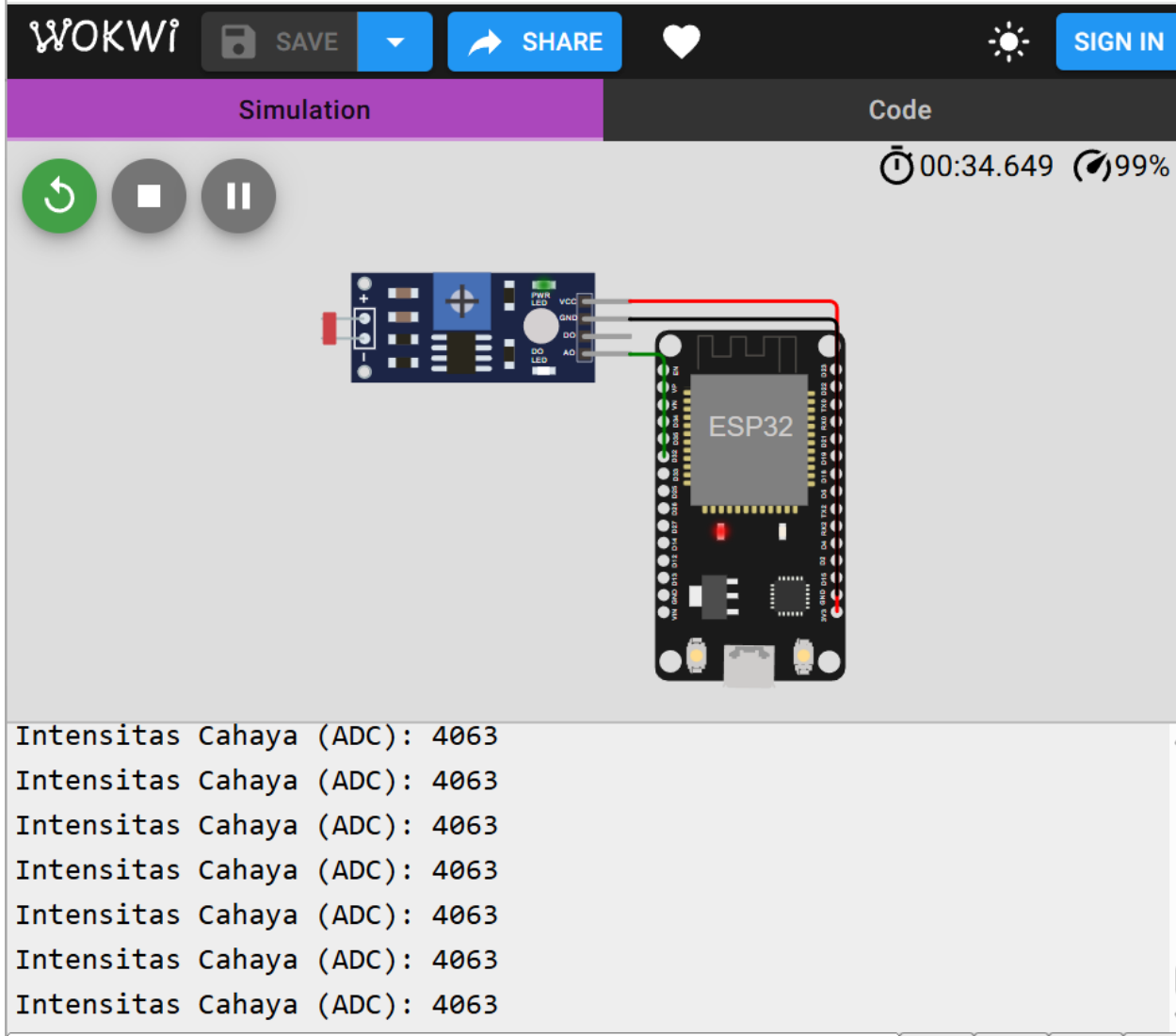
2.6 Jawaban Latihan Mandiri

Latihan 1: Pemahaman Sistem ADC

Soal: Mengapa batas maksimal nilai intensitas cahaya yang terbaca pada Serial Monitor di Praktikum 1 adalah sekitar angka 4063? Kaitkan dengan "Resolusi ADC 10-bit".

Jawaban:

Batas maksimal 4063 terjadi karena mikrokontroler NodeMCU menggunakan sistem ADC (Analog-to-Digital Converter) dengan "Resolusi 10-bit". Dalam sistem komputasi biner, 10-bit merepresentasikan 2^{10} kombinasi nilai, yang menghasilkan total 4063 tingkatan data. Karena perhitungan selalu dimulai dari angka 0, maka rentang nilai pembacaan analog yang dihasilkan adalah dari **0 hingga 4063**.



The screenshot shows a Wokwi simulation environment. At the top, there's a navigation bar with 'WOKWI', 'SAVE', 'SHARE', a heart icon, a sun icon, and 'SIGN IN'. Below this is a tabbed interface with 'Simulation' (active) and 'Code'. The simulation area displays an ESP32 microcontroller board connected to an ADC module. The ADC module has a potentiometer and is connected to the ESP32's pins. The Serial Monitor at the bottom shows the output of the program, which is 'Intensitas Cahaya (ADC): 4063' repeated seven times. The top right of the simulation area shows a timer at 00:34.649 and a battery level at 99%.

Latihan 2: Implementasi Algoritma

Soal: Modifikasi kode pada Praktikum 1 agar program menghitung dan mencetak persentase intensitas cahaya (0-100%).

Jawaban:

Untuk mengubah rentang nilai ADC murni menjadi persentase (0% hingga 100%), rumus matematika dasar yang digunakan adalah:

$$\text{Persentase} = \left(\frac{\text{Nilai ADC}}{\text{Nilai Maksimal ADC}} \right) \times 100$$

Pada platform Arduino, logika pembagian ini dapat disederhanakan menggunakan fungsi bawaan `map()`. Berikut adalah implementasi kodenya pada ESP32 (nilai maksimal 4095):

C++

```
const byte ldrPin = 32;
```

```
void setup() { Serial.begin(115200);  
}
```

```
void loop() {
```

```
    int ldrValue = analogRead(ldrPin);
```

```
    // Mengkonversi nilai ADC ESP32 (0-4095) menjadi Persentase (0-100) int  
    persentase = map(ldrValue, 0, 4095, 0, 100);
```

```
    Serial.print("Nilai ADC: ");
```

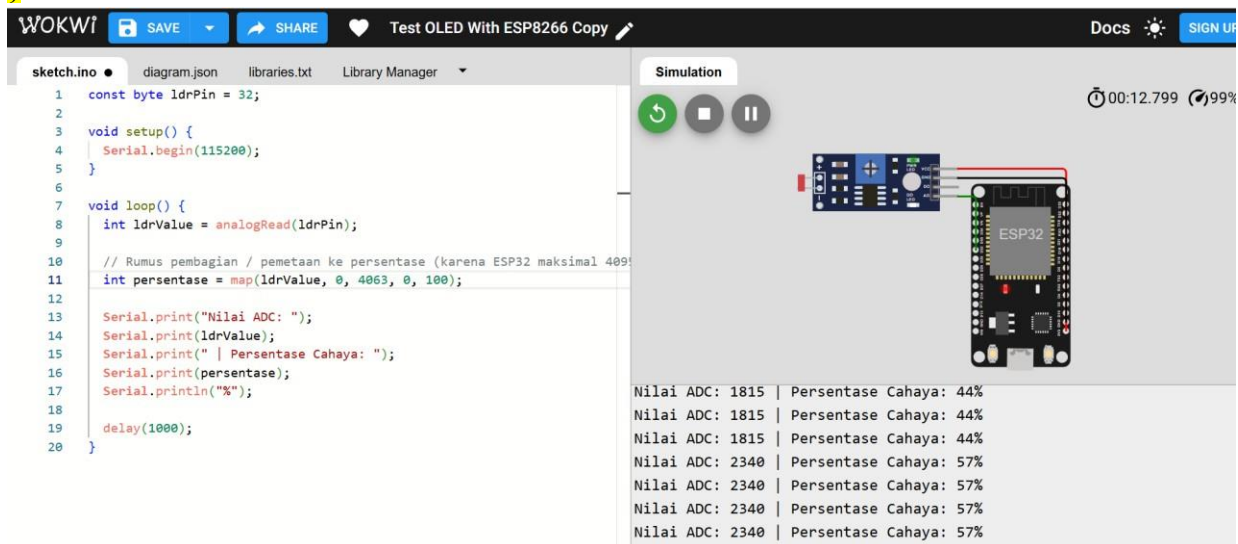
```
    Serial.print(ldrValue);
```

```
    Serial.print(" | Persentase Cahaya: ");
```

```
    Serial.print(persentase); Serial.println("%");
```

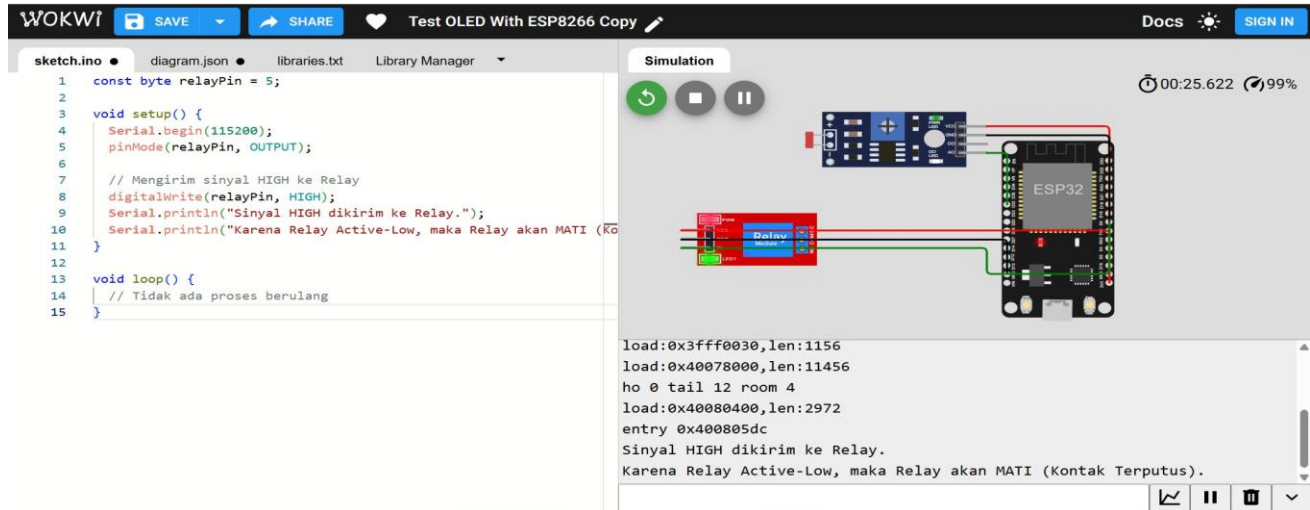
```
    delay(1000);
```

```
}
```



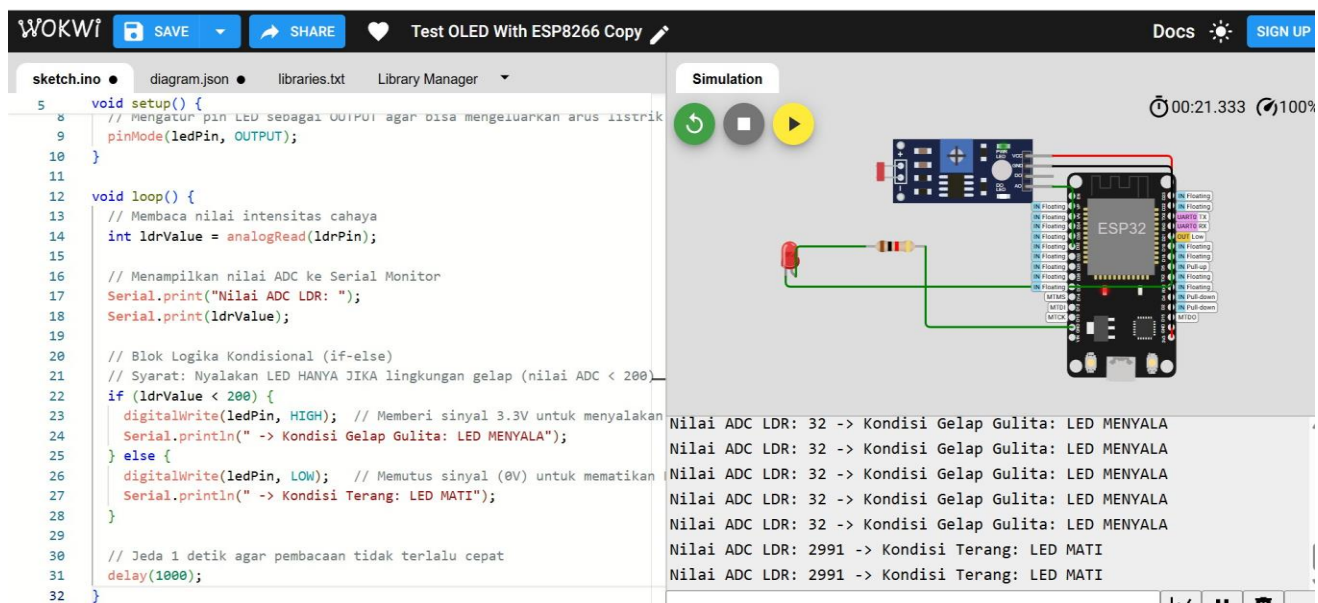
Latihan 3 (Analisis Rangkaian Relay)

Jawaban untuk Laporan: Jika modul Relay bertipe *Active-Low*, aktuator (misal: lampu ruangan) yang terhubung ke relay tersebut akan **MATI (OFF)** saat program mengeksekusi perintah `digitalWrite(relayPin, HIGH)`. Hal ini terjadi karena sifat *Active-Low* bekerja dengan logika terbalik; ia membutuhkan sinyal LOW (0 Volt) untuk mengaktifkan saklar relay, sedangkan sinyal HIGH akan memutus saklar relay



Latihan 4: Pengembangan Logika (LDR + LED)

Soal: Gabungkan logika Praktikum 1 dengan kendali LED. Nyalakan LED HANYA JIKA lingkungan gelap gulita (ambang batas ADC di bawah 200). **Jawaban (Skema & Kode):** Rangkaian ditambahkan satu buah LED pada pin D21 (dihubungkan dengan resistor 220 Ohm ke GND). Logika kendali



2.7 Tugas: Sistem Otomasi Kontrol Lokal Terpadu

Sebuah sistem *Internet of Things (IoT)* harus memiliki kecerdasan dasar pada perangkat lapangannya (*Edge Computing*), sehingga apabila sewaktu-waktu koneksi internet terputus, perangkat masih dapat mengambil keputusan darurat berdasarkan kondisi lingkungan lokalnya.

Skenario Proyek (*Smart Warehouse*): Anda ditugaskan merancang *rule engine* aktuator mandiri. Sistem harus membaca Suhu (DHT) dan Intensitas Cahaya (LDR) secara bersamaan. Modul **Relay 5V** akan disimulasikan sebagai sakelar daya untuk pendingin ruangan (*Exhaust Fan*) atau sistem pencahayaan darurat.

- **Tujuan:** Mampu memadukan akuisisi multi-sensor (Analog LDR dan Digital DHT) dan menggunakan aktuator Relay untuk mengeksekusi keputusan terpadu berbasis kondisi/ambang batas.
- **Tingkat Kesulitan:** Menengah-Lanjut

1. Instruksi Perangkaian:

- Pertahankan sirkuit sensor LDR (di A0) dan DHT (di D7).
- Hubungkan pin **VCC / +5V** dari Modul Relay ke pin **VIN** atau **VU** NodeMCU (mengambil daya murni 5V dari USB). **Jangan menyambungkan modul Relay 5V ke pin 3V3!**
- Hubungkan pin **GND** Relay ke jalur **GND** bersama NodeMCU.
- Hubungkan pin **IN / Signal** Relay ke pin **D6 (GPIO 12)**.
- (Opsional sebagai indikator visual tambahan): Pasang sebuah LED beresistor 220 Ohm pada pin **D1 (GPIO 5)**.

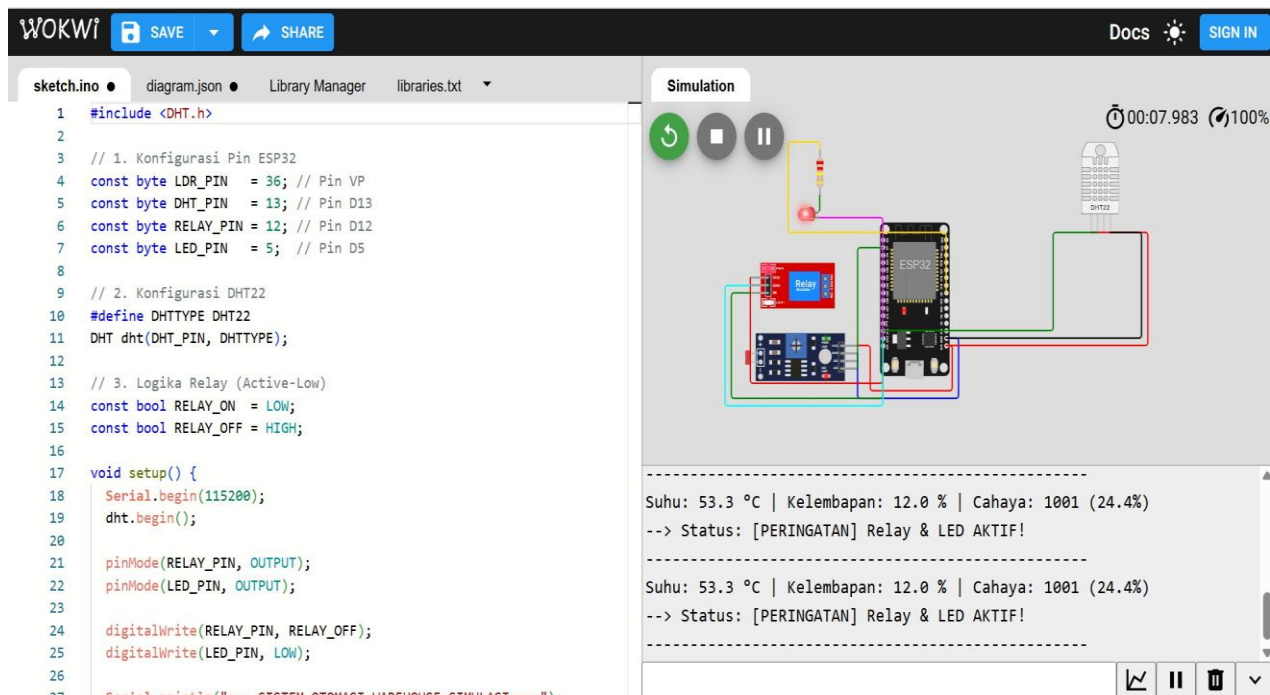
• Instruksi Pemrograman (Aturan/Rule Engine):

- Konfigurasi pin Relay (D6) dan LED (D1) sebagai OUTPUT. (Kirimkan perintah mati di awal fungsi setup() untuk keamanan).
- Buatlah struktur logika perbandingan (if ... else) menggunakan operator logika **OR (||)**.
- Aktifkan Relay dan nyalakan LED indikator** JIKA kondisi lingkungan buruk berikut terpenuhi:
 - Suhu udara terbaca **di atas 34 Celcius** (Terlalu panas).
 - **ATAU** Nilai ADC LDR terbaca **di bawah angka 300** (Terlalu gelap).
- Jika suhu normal dan ruangan cukup cahaya, pastikan Relay dan LED kembali mati.
- Cetak status sistem ke Serial Monitor secara rutin (Contoh: "Kondisi Aman" / "Peringatan: Aktuator Aktif!").

2. Kriteria Keberhasilan:

- Sensor LDR dan DHT berhasil dibaca bersamaan dalam satu *loop* tanpa interupsi atau kegagalan pembacaan (*NaN*).
- Logika kondisi OR beroperasi dengan presisi (cukup salah satu syarat buruk terpenuhi, aktuator menyala).
- Relay bekerja selaras dan logis sesuai dengan sifat modul kelistrikan tersebut (pahami apakah modul Anda bersifat *Active-Low* atau *Active-High*).

UNTUK HASIL DARI TUGAS



```
1 #include <DHT.h>
2
3 // 1. Konfigurasi Pin ESP32
4 const byte LDR_PIN = 36; // Pin VP
5 const byte DHT_PIN = 13; // Pin D13
6 const byte RELAY_PIN = 12; // Pin D12
7 const byte LED_PIN = 5; // Pin D5
8
9 // 2. Konfigurasi DHT22
10 #define DHTTYPE DHT22
11 DHT dht(DHT_PIN, DHTTYPE);
12
13 // 3. Logika Relay (Active-Low)
14 const bool RELAY_ON = LOW;
15 const bool RELAY_OFF = HIGH;
16
17 void setup() {
18   Serial.begin(115200);
19   dht.begin();
20
21   pinMode(RELAY_PIN, OUTPUT);
22   pinMode(LED_PIN, OUTPUT);
23
24   digitalWrite(RELAY_PIN, RELAY_OFF);
25   digitalWrite(LED_PIN, LOW);
26
27   Serial.println("==== SISTEM OTOMATIS WADUKWICE STM32ACT =====");
28 }
```

Suhu: 53.3 °C | Kelembapan: 12.0 % | Cahaya: 1001 (24.4%)
--> Status: [PERINGATAN] Relay & LED AKTIF!

Suhu: 53.3 °C | Kelembapan: 12.0 % | Cahaya: 1001 (24.4%)
--> Status: [PERINGATAN] Relay & LED AKTIF!

Penjelasan hasil simulasi

Pada Serial Monitor terlihat:

Suhu: 53.3 °C | Kelembapan: 12.0 % | Cahaya: 1001 (24.4%)

--> Status: [PERINGATAN] Relay & LED AKTIF!

1. Hasil pembacaan sensor DHT22

Suhu: 53.3 °C

Kelembapan: 12.0 %

Sensor DHT22 berhasil membaca suhu sebesar 53,3°C dan kelembapan sebesar 12%. Nilai suhu tersebut melebihi batas yang ditentukan dalam program, yaitu:

$temp > 34.0$

Karena suhu sebesar 53,3°C lebih tinggi daripada 34°C, maka kondisi ruangan dianggap terlalu panas.

2. Hasil pembacaan sensor LDR

Cahaya: 1001 (24.4%)

Sensor LDR menghasilkan nilai ADC sebesar 1001. Nilai tersebut kemudian dikonversi menjadi persentase:

$float ldrPercent = (ldrValue / 4095.0) * 100.0;$

Perhitungannya:

$1001 / 4095 \times 100 = 24,4\%$

Jadi, nilai pembacaan LDR adalah sekitar 24,4% dari rentang ADC ESP32. Batas

gelap yang digunakan dalam program adalah:

$ldrValue < 1200$

Karena nilai LDR sebesar 1001 lebih kecil daripada 1200, kondisi tersebut juga dianggap sebagai ruangan yang terlalu gelap.

3. Proses logika OR

Kode utama sistem adalah:

```
if (temp > 34.0 || ldrValue < 1200) {
```

Operator || berarti OR atau “atau”. Artinya, cukup salah satu kondisi terpenuhi, sistem akan mengaktifkan aktuator. Pada hasil simulasi:

- Suhu 53,3°C lebih besar dari 34°C → kondisi terpenuhi.
- Nilai LDR 1001 lebih kecil dari 1200 → kondisi terpenuhi. Karena kedua kondisi terpenuhi, program menjalankan perintah: `digitalWrite(RELAY_PIN, RELAY_ON); digitalWrite(LED_PIN, HIGH);` Dengan demikian:
- Relay menjadi aktif.
- LED indikator menyala.
- Serial Monitor menampilkan status peringatan.

Penjelasan diagram rangkaian

Berdasarkan diagram pada screenshot, rangkaian terdiri dari ESP32, sensor DHT22, sensor LDR, modul relay, LED, dan resistor.

1. ESP32

ESP32 berfungsi sebagai pusat pengendali sistem. ESP32 membaca data dari DHT22 dan LDR, kemudian menentukan apakah relay dan LED harus aktif atau mati.

2. Sensor DHT22

Sensor DHT22 digunakan untuk membaca:

- Suhu udara.
- Kelembapan udara.

Pada program, pin data DHT22 dihubungkan ke GPIO13:

```
const byte DHT_PIN = 13;
```

3. Sensor LDR

Sensor LDR digunakan untuk membaca intensitas cahaya ruangan. Output analog LDR dihubungkan ke GPIO36 atau pin VP:

```
const byte LDR_PIN = 36;
```

Pin GPIO36 digunakan karena dapat membaca nilai analog ADC.

4. Modul relay

Relay digunakan sebagai sakelar elektronik untuk mengendalikan perangkat seperti exhaust fan atau lampu darurat.

Pada program:

```
const byte RELAY_PIN = 12;
```

Artinya pin IN relay dikendalikan oleh GPIO12. Relay menggunakan sistem Active-Low, sehingga:

LOW = Relay aktif HIGH =

Relay mati

Pada awal program, relay dimatikan terlebih dahulu:

```
digitalWrite(RELAY_PIN, RELAY_OFF);
```

Hal ini dilakukan agar relay tidak langsung aktif ketika ESP32 baru dinyalakan.

5. LED indikator

LED digunakan sebagai indikator visual. LED dihubungkan ke GPIO5: `const`

```
byte LED_PIN = 5;
```

Jika kondisi lingkungan buruk, LED dinyalakan: `digitalWrite(LED_PIN, HIGH);`

Jika kondisi kembali normal, LED dimatikan:

```
digitalWrite(LED_PIN, LOW);
```

6. Resistor LED

Resistor digunakan untuk membatasi arus yang melewati LED agar LED tidak rusak. Nilai resistor yang digunakan sebaiknya: 220 Ohm

Penjelasan bagian setup()

```
void setup() {  
  Serial.begin(115200);  
  dht.begin();  
  
  pinMode(RELAY_PIN, OUTPUT);  
  pinMode(LED_PIN, OUTPUT);  
  
  digitalWrite(RELAY_PIN, RELAY_OFF);  
  digitalWrite(LED_PIN, LOW);  
  
  Serial.println("=== SISTEM OTOMASI WAREHOUSE SIMULASI ===");  
}
```

Fungsi setup() dijalankan satu kali ketika ESP32 mulai bekerja.

- Serial.begin(115200) memulai komunikasi dengan Serial Monitor.
- dht.begin() mengaktifkan sensor DHT22.
- pinMode() mengatur relay dan LED sebagai output.
- digitalWrite(RELAY_PIN, RELAY_OFF) memastikan relay mati saat awal.
- digitalWrite(LED_PIN, LOW) memastikan LED mati saat awal.
- Serial.println() menampilkan pesan awal sistem.

Penjelasan bagian loop()

```
void loop() {  
  delay(2000);  
  
  int ldrValue = analogRead(LDR_PIN); float  
  temp = dht.readTemperature(); float hum =  
  dht.readHumidity();
```

Hasil pembacaan kemudian diperiksa:

```
if (isnan(temp) || isnan(hum)) {  
  }
```

Pemeriksaan isnan() digunakan untuk memastikan data suhu dan kelembapan valid. Jika pembacaan gagal, program menampilkan pesan error dan melewati proses kontrol pada siklus tersebut.

Analisis hasil pengujian

Hasil pengujian menunjukkan:

Parameter	Hasil	Batas	Keterangan
Suhu	53,3°C	> 34°C	Terlalu panas
Kelembapan	12%	Tidak digunakan dalam logika	Berhasil terbaca
Nilai LDR	1001	< 1200	Kondisi gelap
Persentase cahaya	24,4%	Informasi tambahan	Cahaya rendah
Relay	Aktif	Berdasarkan kondisi	Berhasil
LED	Menyala	Berdasarkan kondisi	Berhasil

Kesimpulan pengujian

Berdasarkan hasil simulasi, sistem Smart Warehouse berhasil membaca sensor DHT22 dan LDR secara bersamaan.

Data suhu terbaca sebesar 53,3°C, kelembapan 12%, dan nilai LDR sebesar 1001.

Suhu melebihi batas 34°C dan nilai LDR berada di bawah batas 1200. Oleh karena itu, kondisi logika OR terpenuhi sehingga relay dan LED aktif. Hal ini membuktikan bahwa sistem mampu mengambil keputusan secara otomatis berdasarkan kondisi lingkungan lokal. Sistem tersebut telah menerapkan konsep Edge Computing, karena proses pembacaan sensor dan pengaktifan aktuator dilakukan langsung oleh ESP32 tanpa membutuhkan koneksi.

